DBMS Lab Sheet 4

Lab Sheet 4: SQL Operators - Arithmetic, Logical, Comparison, Special and Set operators

Run both setup scripts first: the first creates the tables and the second loads the sample data. Outputs shown are for that sample data. INTERSECT and EXCEPT need MySQL 8.0.31 or later; workarounds for older versions are given.

Setup 1: create tables

```
USE CollegeDB;
SET SQL_SAFE_UPDATES = 0;   -- lets UPDATE/DELETE run without a key in WHERE (Workbench)

DROP TABLE IF EXISTS Student, Student_Details, Employee, Department,
                     Product, Customer, Course, Student_Demo;

CREATE TABLE Department (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50) NOT NULL,
    Location  VARCHAR(50)
);
CREATE TABLE Course (
    Course_ID   INT PRIMARY KEY,
    Course_Name VARCHAR(50) NOT NULL,
    Duration    VARCHAR(20),
    Fees        DECIMAL(10,2)
);
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name       VARCHAR(50) NOT NULL,
    Age        INT,
    Course     VARCHAR(30),
    City       VARCHAR(30),
    Dept_ID    INT
);
CREATE TABLE Employee (
    Emp_ID     INT PRIMARY KEY,
    Emp_Name   VARCHAR(50) NOT NULL,
    Department VARCHAR(30),
    Salary     DECIMAL(10,2),
    Email      VARCHAR(80) UNIQUE,
    Join_Date  DATE,
    Manager_ID INT,
    Dept_ID    INT
);
CREATE TABLE Product (
    Product_ID   INT PRIMARY KEY,
    Product_Name VARCHAR(50) NOT NULL,
    Price        DECIMAL(10,2) CHECK (Price > 0),
    Quantity     INT
);
CREATE TABLE Customer (
    Customer_ID   INT PRIMARY KEY,
    Customer_Name VARCHAR(50) NOT NULL,
    Email         VARCHAR(80),
    Phone         VARCHAR(15)
);
```

Setup 2: load sample data

```
USE CollegeDB;
SET SQL_SAFE_UPDATES = 0;
TRUNCATE TABLE Student;   TRUNCATE TABLE Employee;   TRUNCATE TABLE Department;
TRUNCATE TABLE Course;    TRUNCATE TABLE Product;    TRUNCATE TABLE Customer;

INSERT INTO Department VALUES
    (1,'Computer Science','Block A'), (2,'Mathematics','Block B'),
    (3,'Physics','Block C'),          (4,'Commerce','Block D'),
    (5,'Chemistry','Block E');

INSERT INTO Course VALUES
    (101,'BCA','3 Years',90000), (102,'BBA','3 Years',85000),
    (103,'B.Sc','3 Years',60000), (104,'MCA','2 Years',110000),
    (105,'B.Com','3 Years',45000);

INSERT INTO Student VALUES
    (1,'Aarav Sharma',20,'BCA','Meerut',1),
    (2,'Priya Singh',21,'BBA','Delhi',4),
    (3,'Rohan Verma',19,'B.Sc','Meerut',3),
    (4,'Sneha Gupta',22,'MCA','Noida',1),
    (5,'Aman Khan',20,'BCA','Lucknow',1),
    (6,'Kavya Jain',21,'B.Sc','Delhi',2),
    (7,'Rahul Yadav',23,'MCA','Meerut',NULL),
    (8,'Neha Tyagi',20,'B.Com','Ghaziabad',4);

INSERT INTO Employee VALUES
    (1,'Anil Mehta','Computer Science',85000,'anil@college.edu','2015-06-15',NULL,1),
    (2,'Sunita Rao','Computer Science',60000,'sunita@college.edu','2018-07-01',1,1),
    (3,'Vikram Joshi','Mathematics',70000,'vikram@college.edu','2016-08-20',NULL,2),
    (4,'Pooja Malik','Mathematics',45000,'pooja@college.edu','2020-01-10',3,2),
    (5,'Manoj Tiwari','Commerce',52000,'manoj@college.edu','2019-03-25',NULL,4),
    (6,'Deepa Nair','Commerce',38000,'deepa@college.edu','2021-09-05',5,4),
    (7,'Imran Ali','Physics',65000,'imran@college.edu','2017-11-12',NULL,3),
    (8,'Ritu Saxon','Computer Science',47000,'ritu@college.edu','2022-02-14',1,1);

INSERT INTO Product VALUES
    (1,'Laptop',55000.00,10), (2,'Mouse',500.00,50), (3,'Keyboard',1200.00,0),
    (4,'Monitor',9500.00,15), (5,'USB Drive',650.50,0);

INSERT INTO Customer VALUES
    (1,'Ankit Arora','ankit@example.com','9876543210'),
    (2,'Meena Kapoor','meena@example.com','9876501234'),
    (3,'Sandeep Rana','sandeep@example.com','9812345678');
```

Q1. Perform addition of two numbers using +.

```
SELECT 15 + 5 AS Addition;
```

Output:

Addition
20

Q2. Perform subtraction using -.

```
SELECT 20 - 8 AS Subtraction;
```

Output:

Subtraction
12

Q3. Perform multiplication using *.

```
SELECT 6 * 7 AS Multiplication;
```

Output:

Multiplication
42

Q4. Perform division using /.

```
SELECT 22 / 7 AS Division, 22 DIV 7 AS Integer_Division, 22 % 7 AS Remainder;
```

Output:

Division	Integer_Division	Remainder
3.1429	3	1

/ gives a decimal result, DIV truncates to an integer, % returns the remainder.

Q5. Calculate the total amount as Price × Quantity.

```
SELECT Product_Name, Price, Quantity, Price * Quantity AS Total_Amount
FROM Product;
```

Output:

Product_Name	Price	Quantity	Total_Amount
Laptop	55000.00	10	550000.00
Mouse	500.00	50	25000.00
Keyboard	1200.00	0	0.00
Monitor	9500.00	15	142500.00
USB Drive	650.50	0	0.00

Q6. Demonstrate AND with two conditions.

```
SELECT Name, Age, City FROM Student
WHERE Age >= 21 AND City = 'Delhi';
```

Output:

Name	Age	City
Priya Singh	21	Delhi
Kavya Jain	21	Delhi

Q7. Demonstrate OR with two conditions.

```
SELECT Name, City FROM Student
WHERE City = 'Meerut' OR City = 'Noida';
```

Output:

Name	City
Aarav Sharma	Meerut
Rohan Verma	Meerut
Sneha Gupta	Noida
Rahul Yadav	Meerut

Q8. Demonstrate NOT with a condition.

```
SELECT Name, Course FROM Student WHERE NOT Course = 'BCA';
```

NOT reverses the condition; it returns the same rows as Course <> 'BCA'.

Q9. Find students belonging to a specific course AND city.

```
SELECT * FROM Student WHERE Course = 'BCA' AND City = 'Meerut';
```

Output:

Student_ID	Name	Age	Course	City	Dept_ID
1	Aarav Sharma	20	BCA	Meerut	1

Q10. Find employees belonging to either of two departments.

```
SELECT Emp_Name, Department FROM Employee
WHERE Department = 'Computer Science' OR Department = 'Commerce';
```

Output:

Emp_Name	Department
Anil Mehta	Computer Science
Sunita Rao	Computer Science
Manoj Tiwari	Commerce
Deepa Nair	Commerce
Ritu Saxon	Computer Science

Q11. Demonstrate the = operator.

```
SELECT Name, Course FROM Student WHERE Course = 'BCA';
```

Output:

Name	Course
Aarav Sharma	BCA
Aman Khan	BCA

Q12. Demonstrate the > operator.

```
SELECT Name, Age FROM Student WHERE Age > 20;
```

Output:

Name	Age
Priya Singh	21
Sneha Gupta	22
Kavya Jain	21
Rahul Yadav	23

Q13. Demonstrate the < operator.

```
SELECT Emp_Name, Salary FROM Employee WHERE Salary < 50000;
```

Output:

Emp_Name	Salary
Pooja Malik	45000.00
Deepa Nair	38000.00
Ritu Saxon	47000.00

Q14. Demonstrate >= and <=.

```
SELECT Name, Age FROM Student WHERE Age >= 20 AND Age <= 21;
```

Output:

Name	Age
Aarav Sharma	20
Priya Singh	21
Aman Khan	20
Kavya Jain	21
Neha Tyagi	20

Q15. Demonstrate the <> / != operator.

```
SELECT Name, City FROM Student WHERE City <> 'Delhi';
SELECT Name, City FROM Student WHERE City != 'Delhi';   -- same result
```

Q16. Demonstrate BETWEEN for a salary range.

```
SELECT Emp_Name, Salary FROM Employee
WHERE Salary BETWEEN 45000 AND 65000;
```

Output:

Emp_Name	Salary
Sunita Rao	60000.00
Pooja Malik	45000.00
Manoj Tiwari	52000.00
Imran Ali	65000.00
Ritu Saxon	47000.00

BETWEEN is inclusive of both end values.

Q17. Demonstrate IN for selecting multiple cities.

```
SELECT Name, City FROM Student
WHERE City IN ('Meerut', 'Delhi', 'Noida');
```

Six students match: Aarav, Priya, Rohan, Sneha, Kavya and Rahul.

Q18. Demonstrate NOT IN.

```
SELECT Name, City FROM Student
WHERE City NOT IN ('Meerut', 'Delhi', 'Noida');
```

Output:

Name	City
Aman Khan	Lucknow
Neha Tyagi	Ghaziabad

Q19. Demonstrate LIKE for names beginning with a particular letter.

```
SELECT Name FROM Student WHERE Name LIKE 'A%';   -- starts with A
SELECT Name FROM Student WHERE Name LIKE '%a';   -- ends with a
SELECT Name FROM Student WHERE Name LIKE '_a%';  -- second letter is a
```

Output:

Name (starts with A)
Aarav Sharma
Aman Khan

% matches any number of characters and _ matches exactly one.

Q20. Demonstrate IS NULL and IS NOT NULL.

```
SELECT Name FROM Student WHERE Dept_ID IS NULL;
SELECT Emp_Name FROM Employee WHERE Manager_ID IS NOT NULL;
```

Output:

Name (no department)
Rahul Yadav

Comparing with = NULL never works; NULL must be tested with IS NULL or IS NOT NULL.

Q21. Demonstrate UNION using two compatible queries.

```
SELECT Dept_ID FROM Student  WHERE Dept_ID IS NOT NULL
UNION
SELECT Dept_ID FROM Employee
ORDER BY Dept_ID;
```

Output:

Dept_ID
1
2
3
4

UNION removes duplicate rows.

Q22. Demonstrate UNION ALL.

```
SELECT COUNT(*) AS Total_Rows FROM (
    SELECT Dept_ID FROM Student WHERE Dept_ID IS NOT NULL
    UNION ALL
    SELECT Dept_ID FROM Employee
) AS t;
```

Output:

Total_Rows
15

UNION ALL keeps duplicates: 7 student rows + 8 employee rows = 15.

Q23. Demonstrate INTERSECT where supported by the DBMS.

```
SELECT Dept_ID FROM Student
INTERSECT
SELECT Dept_ID FROM Employee
ORDER BY Dept_ID;

-- workaround for MySQL older than 8.0.31
SELECT DISTINCT Dept_ID FROM Student
WHERE Dept_ID IN (SELECT Dept_ID FROM Employee);
```

Output:

Dept_ID
1
2
3
4

INTERSECT and EXCEPT need MySQL 8.0.31 or later.

Q24. Demonstrate EXCEPT/MINUS where supported.

```
-- departments that have no students
SELECT Dept_ID FROM Department
EXCEPT
SELECT Dept_ID FROM Student WHERE Dept_ID IS NOT NULL;

-- workaround for older versions
SELECT Dept_ID FROM Department d
WHERE NOT EXISTS (SELECT 1 FROM Student s WHERE s.Dept_ID = d.Dept_ID);
```

Output:

Dept_ID
5

Oracle calls this operator MINUS; MySQL and SQL Server call it EXCEPT.

Q25. Solve a problem using arithmetic, logical, comparison and special operators together.

```
-- Employees of Computer Science or Commerce who earn at least 40000, have a
-- manager, and whose name does not start with D: show a 10% revised salary.
SELECT Emp_Name,
       Salary,
       Salary * 1.10 AS Revised_Salary
FROM Employee
WHERE Department IN ('Computer Science', 'Commerce')
  AND Salary >= 40000
  AND Manager_ID IS NOT NULL
  AND NOT Emp_Name LIKE 'D%'
ORDER BY Salary DESC;
```

Output:

Emp_Name	Salary	Revised_Salary
Sunita Rao	60000.00	66000.0000
Ritu Saxon	47000.00	51700.0000

Combines: * (arithmetic), IN, IS NOT NULL, LIKE (special), >= (comparison), AND and NOT (logical).